

Manual de funcionamiento instalación y operación de SmartBin

Aplicación AllpaVision y basurero inteligente con ESP32

Edición del 4 de octubre de 2026

Este manual explica cómo funciona el prototipo, cómo se comunican sus componentes, cómo trasladarlo a otra computadora Windows y cómo prepararlo para una exposición. La aplicación clasifica residuos mediante un modelo de inteligencia artificial, abre el compartimento correspondiente y acredita puntos después del ciclo de apertura y cierre.

El sistema utiliza una computadora como servidor local y un ESP32 como controlador físico. El ESP32 no ejecuta el modelo de inteligencia artificial. El montaje no tiene sensor de depósito: la confirmación significa que el programa terminó el ciclo temporizado, no que detectó un residuo dentro del basurero.

1 Alcance y componentes

La guía principal utiliza Flutter Web en Chrome, PostgreSQL local, Mosquitto local y un ESP32 clásico. Las instrucciones para Android se incluyen como alternativa. Los comandos están escritos para PowerShell. La carpeta de ejemplo en la computadora nueva será C:\SmartBin\practicas; debe sustituirse si se elige otra ubicación.

Componente	Responsabilidad
Flutter y Dart	Interfaz, registro, inicio de sesión, cámara o galería, resultados e historial
Node.js y Express	Autenticación, reglas de reciclaje, depósitos, puntos y envío MQTT
Prisma y PostgreSQL	Persistencia de usuarios, depósitos y movimientos de puntos
Python y FastAPI	Recibe una imagen y devuelve la clasificación
TensorFlow y Keras	Cargan y ejecutan el modelo EfficientNetB0 entrenado
OpenCV y NumPy	Decodifican y preparan la imagen para inferencia
Mosquitto	Broker que entrega mensajes MQTT entre backend y ESP32
ESP32 y firmware Arduino	Recibe la orden, selecciona el canal y controla el ciclo
PCA9685 y cuatro FT5325M	Generan señales PWM y accionan las puertas

No se requiere ejecutar esp32-simulator cuando el dispositivo físico está activo. El simulador es una alternativa para ensayos sin hardware.

2 Arquitectura del sistema

Se utiliza una arquitectura cliente servidor con separación de responsabilidades y comunicación por eventos para los actuadores. El backend Node es la API principal. FastAPI es un servicio independiente de inferencia. No se requiere Docker ni un servicio en la nube para la exposición local.

2 1 Recorrido de datos

Usuario -> Flutter Web -> HTTP y JWT -> Backend Node
Backend Node -> HTTP con imagen -> FastAPI -> EfficientNetB0
FastAPI -> clase y confianza -> Backend Node -> PostgreSQL
Backend Node -> MQTT OPEN -> Mosquitto -> ESP32
ESP32 -> I2C -> PCA9685 -> servo de la puerta
ESP32 -> MQTT confirmacion -> Mosquitto -> Backend Node
Backend Node -> transaccion de puntos -> PostgreSQL
Flutter -> consulta de perfil e historial -> Backend Node

HTTP transporta solicitudes y respuestas entre aplicación y APIs. MQTT desacopla el backend del controlador físico: ambos se conectan al broker y publican o se suscriben a canales. I2C conecta el ESP32 con el PCA9685 mediante SDA y SCL. PWM posiciona los servos.

2 2 Direcciones y puertos

Servicio	Dirección de referencia	Uso
PostgreSQL	localhost:5432	Base de datos local
Mosquitto	TCP 1883	Backend y ESP32
FastAPI	http://127.0.0.1:8000	Inferencia y documentación
Backend Node	http://127.0.0.1:3000/api/v1	API consumida por Flutter
Flutter Web	http://localhost:PUERTO	El puerto se informa al ejecutar Flutter

127.0.0.1 significa el equipo que ejecuta el proceso. El backend puede usar esa dirección para acceder a Mosquitto y FastAPI en la misma computadora. El ESP32 debe usar la IPv4 local de la computadora, por ejemplo 192.168.0.100. El valor es un ejemplo y debe obtenerse con ipconfig.

Mantener FastAPI y PostgreSQL locales reduce la exposición de servicios. Para este montaje, el ESP32 solo necesita llegar al puerto MQTT 1883.

3 Funcionamiento de la aplicación

3 1 Cuenta y sesión

El usuario registra nombre, correo y contraseña. El backend genera un código de verificación por correo. Después de verificar la cuenta, el usuario inicia sesión. El backend entrega un JWT que permite consultar el perfil, clasificar y consultar el historial. Las contraseñas se protegen con bcrypt; no se almacenan como texto legible.

La entrega de códigos necesita acceso a Internet y una configuración SMTP válida. Para la exposición conviene llevar una cuenta ya registrada y verificada. La clasificación local no requiere una API de IA externa, pero la instalación inicial sí necesita descargar herramientas y dependencias.

3 2 Clasificación y decisión

1. El usuario captura una fotografía o selecciona una imagen.
2. Flutter envía la imagen al backend autenticado, utilizando el campo multipart image.
3. Node remite la imagen a FastAPI.
4. FastAPI decodifica la imagen con OpenCV, convierte BGR a RGB, redimensiona a 224 por 224 y crea el tensor de entrada.
5. EfficientNetB0 devuelve probabilidades. La API obtiene la clase de mayor probabilidad y un ranking de resultados.
6. Node compara el resultado con las reglas de residuos. La apertura requiere una clase reciclable y confianza mayor o igual a 0.70.
7. Si la apertura procede, crea un depósito PENDING con cero puntos y publica OPEN por MQTT.
8. Flutter muestra clase, confianza, estado y si el comando fue publicado.

El porcentaje de confianza es una salida del modelo; no certifica por sí solo que la clasificación sea correcta. Para una exposición se debe ensayar con objetos e iluminación reales.

3 3 Apertura y acreditación

El ESP32 escucha smartbin/bin-01/commands/open. Valida el JSON, convierte el compartimento en un canal, abre el servo seleccionado y espera 5000 ms. Después ordena el cierre y deja 1000 ms para el regreso. El tiempo se controla con millis para mantener MQTT atendido.

Al terminar, publica el identificador del depósito en smartbin/bin-01/events/deposit-confirmed. Node actualiza el depósito y los puntos dentro de una transacción de PostgreSQL. Las confirmaciones repetidas del mismo depósito no deben sumar puntos nuevamente. La interfaz puede necesitar volver a consultar el perfil o abrir otra vez el historial para mostrar el saldo nuevo.

La confirmación temporizada no mide la posición real del servo ni la presencia del residuo. Si un servo está desconectado, el programa puede terminar su temporizador igualmente. Esta limitación debe explicarse durante la demostración.

4 Reglas de materiales y datos

Clase del modelo	Compartimento MQTT	Canal	Puntos
plastic	plastic	0	10
metal	metal	1	20
paper	paper_cardboard	2	5
cardboard	paper_cardboard	2	5
glass	glass	3	15
trash	Sin apertura	Ninguno	0

El canal 1 corresponde a lata en el montaje, pero la clase del modelo es metal: otros objetos metálicos también pueden activar ese compartimento. Papel y cartón comparten el canal 2.

Estado	Significado operativo
PENDING	Clasificación aceptada; falta confirmación del ciclo
CONFIRMED	Backend recibió la confirmación y acreditó puntos
CANCELLED	La clasificación no cumple las reglas de apertura
FAILED	Se produjo un fallo registrado, por ejemplo al publicar la orden

Una orden MQTT publicada no equivale a una puerta abierta. En esta versión las órdenes OPEN no son retenidas: una placa desconectada puede perderlas. Tampoco hay cancelación automática de todos los pendientes ni reproducción automática al reiniciar. No enviar otra clasificación mientras una puerta está ocupada: el firmware inicial procesa un ciclo a la vez y no mantiene una cola de órdenes.

Prisma organiza tres entidades principales: User contiene identidad, verificación y saldo; Deposit relaciona usuario, predicción, clase, confianza y estado; PointMovement registra la cantidad y motivo de cada movimiento. La base permite relacionar los puntos con el depósito que los originó.

5 Montaje físico

5 1 Materiales

ESP32 clásico identificado por la herramienta como ESP32-D0WD-V3; módulo PCA9685; cuatro servos FT5325M; cable USB de datos; fuente y convertidor adecuados; cables de alimentación, conectores y

estructura de puertas. La protoboard sirve para interconexiones de señal, pero la corriente de motores debe ir por cableado de alimentación adecuado.

5 2 Cableado

Origen	Destino
ESP32 GPIO 21	PCA9685 SDA
ESP32 GPIO 22	PCA9685 SCL
ESP32 3.3 V	PCA9685 VCC
ESP32 GND	PCA9685 GND
Fuente OUT positivo	PCA9685 V+ de alimentación de servos
Fuente OUT negativo	PCA9685 GND
Servo plástico	Canal 0 señal, V+ y GND
Servo lata	Canal 1 señal, V+ y GND
Servo papel	Canal 2 señal, V+ y GND
Servo vidrio	Canal 3 señal, V+ y GND

VCC alimenta la lógica del módulo; V+ alimenta los servos. No unir V+ con 3.3 V del ESP32. En la configuración de exposición, el ESP32 se alimenta por USB y los motores mediante su fuente. Mantener una tierra común permanente. Colocar el interruptor en la alimentación positiva de motores y no utilizarlo para cortar la tierra común. Desenergizar USB y fuente antes de recablear.

La ficha FT5325M consultada especifica un servo de 180 grados y pulsos de 500 a 2500 microsegundos. Presenta datos de funcionamiento a 6 y 7.4 V, con corriente de bloqueo de 3 y 3.5 A por servo respectivamente. Esto explica por qué una fuente pequeña puede funcionar sin carga y fallar al mover las puertas. Dimensionar fuente, convertidor y cables para los picos y cargas previstos; no mantener un motor bloqueado. Verificar también el voltaje admitido por el módulo PCA9685 concreto antes de elegir la alimentación. Medir la salida del convertidor sin el montaje conectado antes de energizarlo y conservar GND común.

5 3 Parámetros de movimiento

```
const bool SOLO_PRUEBA_MQTT = false;
const int CERRADO_US = 1250;
const int ABIERTO_US = 1800;
```

1800 microsegundos es un punto de partida para la apertura ampliada. Usar el valor que realmente se validó en cada puerta. Los valores no son grados y dependen del montaje. Mantener los 5000 ms de apertura y 1000 ms de regreso del firmware integrado. Si cada puerta necesita una calibración distinta, se requerirán parámetros por canal; un único valor global no garantiza el mismo ángulo físico.

6 Preparación para trasladar el proyecto

6 1 Qué copiar

Copiar app, util, modelo, backend-node, flutter-app, documentacion y los archivos de dependencias. Incluir modelo/modelo_residuos.keras y modelo/clases.json. Conservar package-lock.json, pubspec.lock y backend-node/prisma/migrations. Dataset y notebooks son útiles para entrenamiento, pero no son necesarios para ejecutar una inferencia con el modelo guardado.

Guardar desde Arduino IDE el sketch MQTT que se cargó y funcionó, con su carpeta del mismo nombre, por ejemplo smartbin_esp32/smartbin_esp32.ino. La carpeta esp32-firmware/prueba_servos del repositorio contiene una prueba autónoma; no reemplaza el firmware MQTT. Confirmar que se transporta el sketch correcto y no solo el de movimiento secuencial.

No copiar .venv, node_modules, .dart_tool ni build como método de instalación. Esas carpetas se recrean en la computadora destino. Copiar las credenciales de .env solo mediante un medio privado o crearlas de nuevo; no incluir contraseñas reales en una entrega pública. El firmware puede contener contraseña Wi-Fi y también debe tratarse como información privada.

6 2 Registrar el entorno de origen

Anotar versiones antes de mover el sistema. En la computadora original se utilizó Python 3.13.5 de C:\Python313; el entorno .venv creado con Anaconda produjo errores de DLL. No reutilizar ese entorno defectuoso en la nueva computadora.

```
& "C:\Python313\python.exe" --version
& "C:\Python313\python.exe" -m pip freeze |
  Set-Content -Encoding utf8 requirements-equipos-origen.txt
node --version
npm.cmd --version
flutter --version
```

El archivo freeze incluye paquetes del entorno original y puede contener dependencias ajenas al proyecto. Revisarlo antes de usarlo como una lista de instalación. Para inferencia, el apartado 8 ofrece una selección de dependencias relevantes observadas en ese intérprete. El equipo origen tenía Node 22.16.0; registrar también la versión de Flutter y PostgreSQL realmente usadas.

6 3 Datos existentes o instalación vacía

Decidir antes de instalar: una base nueva comienza sin cuentas, historial ni puntos. Para conservarlos, realizar un respaldo de la base original y restaurarlo en destino. No basta con copiar el código. El apartado 9 explica ambas rutas.

7 Herramientas en la computadora nueva

Utilizar Windows de 64 bits y una cuenta con permisos para instalar software. Instalar las herramientas desde sus páginas oficiales y reiniciar las terminales después de modificar PATH.

Herramienta	Instalación y verificación
Python	Instalar Python 3.13 de 64 bits; comprobar con <code>py -3.13 --version</code>
Runtime Visual C++	Instalar el redistribuible x64 requerido por TensorFlow en Windows
Node.js	Instalar una versión compatible; preferir misma rama 22 para reproducir el entorno y ensayar antes de migrar de rama
PostgreSQL y pgAdmin	Instalar servidor local; anotar versión, contraseña y puerto 5432
Mosquitto	Instalar Windows x64 y las herramientas <code>mosquitto_pub</code> y <code>mosquitto_sub</code>
Flutter y Git	Extraer Flutter, añadir su carpeta <code>bin</code> a PATH e instalar Git
Chrome	Navegador para Flutter Web y permisos de cámara
Arduino IDE	Gestor de placas ESP32 y bibliotecas del firmware
VS Code	Editor y terminales PowerShell para operar el proyecto

Descargas: Python <https://www.python.org/downloads/windows/> ; Node <https://nodejs.org/en/download> ; PostgreSQL <https://www.postgresql.org/download/windows/> ; Mosquitto <https://mosquitto.org/download/> ; Flutter <https://docs.flutter.dev/install/manual> ; Arduino <https://www.arduino.cc/en/software/> .

El proyecto Flutter declara Dart ^3.12.2. Elegir una versión de Flutter que incluya un Dart compatible con esa restricción. No instalar Visual Studio para compilar una aplicación Windows si la exposición utilizará solamente Chrome; atender los avisos de flutter doctor relevantes para web. Android sí necesita su SDK y configuración adicional.

```
py -3.13 --version
node --version
npm.cmd --version
git --version
flutter --version
flutter doctor
flutter devices
```

Chrome debe figurar como dispositivo. Si un comando no se reconoce, revisar PATH y abrir una terminal nueva. No reducir las restricciones del proyecto solo para ocultar incompatibilidades.

8 Instalación del servicio Python

8.1 Crear un entorno nuevo

Copiar el proyecto a C:\SmartBin\practicas. No copiar el entorno Anaconda de la computadora original. Crear un entorno propio con Python instalado en destino; no se requiere activar el entorno para ejecutar sus comandos.

```
cd C:\SmartBin\practicas
py -3.13 -m venv .venv-expo
.\.venv-expo\Scripts\python.exe -m pip install --upgrade pip
```

8.2 Dependencias de inferencia

El requirements.txt del repositorio contiene además herramientas de entrenamiento y versiones que difieren del intérprete que funcionó. Para reproducir la ruta de inferencia de la computadora original, crear un archivo requirements-inferencia.txt con lo siguiente y probarlo en destino:

```
tensorflow==2.21.0
keras==3.15.0
numpy==2.3.3
opencv-python==5.0.0.93
fastapi==0.136.1
uvicorn==0.46.0
pydantic==2.13.4
python-multipart==0.0.32
matplotlib==3.11.2
```

Estas versiones se observaron en el intérprete de origen; no sustituyen la validación en el nuevo hardware. Matplotlib se importa desde util/modelo.py aunque la API no dibuje gráficas. Las dependencias transitivas se resuelven con pip. Si no existe un paquete para la plataforma o aparece un conflicto, conservar el error y resolver la compatibilidad antes de continuar; no mezclar paquetes de Anaconda y otro Python.

```
.\.venv-expo\Scripts\python.exe -m pip install -r requirements-inferencia.txt
.\.venv-expo\Scripts\python.exe -m pip check
.\.venv-expo\Scripts\python.exe -c "import tensorflow as tf; print(tf.__version__)"
.\.venv-expo\Scripts\python.exe -c "import cv2, matplotlib, fastapi, uvicorn; print('Dependencias OK')"
```

8.3 Verificar el modelo y la API

```
Test-Path .\modelo\modelo_residuos.keras
Test-Path .\modelo\clases.json
.\.venv-expo\Scripts\python.exe -m uvicorn app.api.main:app --host 127.0.0.1 --port 8000
```

Ambos Test-Path deben responder True. Esperar Application startup complete y abrir <http://127.0.0.1:8000/docs> y <http://127.0.0.1:8000/api/v1/health>. El health debe indicar que el modelo está cargado. La primera predicción puede tardar más que las siguientes. Detener con Ctrl+C si se trata solo de una validación de instalación.

En la computadora original el comando correcto usaba C:\Python313\python.exe; esa ruta no es universal. En destino utilizar siempre .venv-expo o la ruta explícita del intérprete que se haya probado. Un mensaje Will watch for changes no confirma que TensorFlow ni el modelo hayan cargado.

9 PostgreSQL y backend Node

9 1 Crear una base nueva

Abrir pgAdmin y conectar al servidor local. En Databases seleccionar Create y Database; escribir reciclaje_inteligente. No es necesario exponer PostgreSQL al Wi-Fi. Guardar la contraseña del usuario de PostgreSQL para la conexión del backend.

9 2 Configurar el backend

En backend-node, crear .env desde .env.example solo si no existe. Si ya existe, editarlo sin sobrescribir sus valores.

```
cd C:\SmartBin\practicass\backend-node
if (!(Test-Path .env)) { Copy-Item .env.example .env }

PORT=3000
NODE_ENV=development
AI_API_URL=http://127.0.0.1:8000
AI_API_TIMEOUT_MS=30000
CORS_ORIGIN=*
DATABASE_URL="postgresql://postgres:CLAVE@localhost:5432/reciclaje_inteligente?schema=public"
JWT_SECRET=REEMPLAZAR_POR_UN_SECRETO_ALEATORIO_DE_32_O_MAS_CARACTERES
JWT_EXPIRES_IN=8h
DEVICE_API_KEY=REEMPLAZAR_POR_OTRO_SECRETO_ALEATORIO_DE_32_O_MAS_CARACTERES
MQTT_URL=mqtt://127.0.0.1:1883
MQTT_DEVICE_ID=bin-01
SMTP_USER=correo@gmail.com
SMTP_PASS=CONTRASENA_DE_APLICACION_REAL
SMTP_FROM_NAME=SmartBin
EMAIL_VERIFICATION_EXPIRES_MINUTES=10
```

Reemplazar todos los marcadores. Usar secretos aleatorios, no el texto de ejemplo. Si la contraseña de PostgreSQL contiene caracteres reservados de URL, codificarlos para DATABASE_URL. SMTP_PASS debe ser una contraseña de aplicación válida del proveedor; para Gmail no usar directamente la contraseña habitual de la cuenta. El backend valida las variables y puede detenerse aunque la exposición no registre usuarios nuevos.

9 3 Instalar y aplicar migraciones

```
npm.cmd ci
npx.cmd prisma generate
npx.cmd prisma migrate deploy
```

Usar npm.cmd ci cuando package-lock.json esté presente y coincida con package.json. Si no existe lock, npm.cmd install es la alternativa, pero registrar las versiones resultantes. Las migraciones crean el esquema; no importan usuarios existentes.

9 4 Conservar cuentas y puntos existentes

En la computadora original, abrir pgAdmin, seleccionar la base y Backup. Elegir formato Custom y guardar reciclaje_inteligente.backup. Conservarlo de forma privada porque contiene datos personales. Si se usan herramientas de línea de comandos, pg_dump debe ser compatible con la versión del servidor origen.

En destino crear una base vacía y utilizar Restore de pgAdmin con el archivo. Revisar las opciones de propietario y permisos si cambian los roles; los datos deben quedar accesibles al usuario de DATABASE_URL. No mezclar una restauración completa con tablas ya creadas sin planificar conflictos. Después de restaurar ejecutar prisma generate y prisma migrate deploy para aplicar solo las migraciones faltantes.

Verificar que existan users, deposits, point_movements y el historial de migraciones. Iniciar sesión con una cuenta importada y comprobar el saldo. Si se decide empezar con base nueva, registrar y verificar una cuenta antes de la expo.

10 Configuración de MQTT y red

10 1 Una sola instancia de Mosquitto

En la raíz crear mosquitto-local.conf:

```
listener 1883 0.0.0.0
allow_anonymous true
```

Esta configuración es para una demostración en red privada de confianza. Acepta clientes sin contraseña. No publicar el puerto en Internet. Para despliegue permanente se requieren autenticación, permisos por canal y revisión de la confirmación; DEVICE_API_KEY no autentica por sí sola los mensajes MQTT actuales.

Antes de iniciar, comprobar quién usa el puerto:

```
Get-NetTCPConnection -LocalPort 1883 -State Listen |
Select-Object LocalAddress, OwningProcess
```

Si no aparece un listener, iniciar Mosquitto. Si hay uno, identificarlo con Get-Process -Id NUMERO. Si el instalador activó un servicio, detenerlo en services.msc antes de ejecutar la instancia manual. No detener procesos sin identificarlos. Dos instancias, una en localhost y otra en la red, pueden recibir mensajes diferentes: este problema impidió la entrega al ESP32 durante la puesta en marcha.

```
cd C:\SmartBin\practicas
& "C:\Program Files\mosquitto\mosquitto.exe" -c ".\mosquitto-local.conf" -v
```

Mantener la terminal abierta. En Windows Firewall permitir Mosquitto o TCP 1883 entrante para el perfil privado utilizado por la red. No desactivar todo el firewall. No es necesario abrir 5432 ni 8000 para el ESP32.

10 2 Direcciones al cambiar de computadora

Ejecutar ipconfig y localizar la IPv4 del adaptador conectado a la misma red que el ESP32. No elegir un adaptador virtual o VPN. Actualizar MQTT_HOST en el sketch con esa IPv4. Se recomienda reservar la IP en el router o verificarla antes de cada exposición.

El backend usa MQTT_URL=mqtt://127.0.0.1:1883 porque se ejecuta en el mismo equipo del broker. El identificador MQTT_DEVICE_ID debe ser bin-01 para coincidir con los canales del firmware. La red debe permitir comunicación entre clientes; las redes de invitados o con aislamiento pueden impedirla.

10 3 Diagnóstico de entrega

Con SOLO_PRUEBA_MQTT=true y el ESP32 por USB, ejecutar en otra terminal:

```
& "C:\Program Files\mosquitto\mosquitto_pub.exe" -h 127.0.0.1 -p 1883 -t "smartbin/bin-01/commands/open" -m "PRUEBA" -q 1 -d
```

Arduino debe imprimir Mensaje recibido, PRUEBA y ERROR JSON invalido. El error de JSON es esperado: el texto sirve para verificar transporte sin mover motores. PUBACK en el publicador demuestra recepción por el broker; revisar Sending PUBLISH to esp32 en Mosquitto y el monitor serie para comprobar entrega al dispositivo.

11 Preparación y carga del ESP32

1. Instalar Arduino IDE en destino si se necesitará cambiar Wi-Fi, dirección o calibración.
2. En el Gestor de placas instalar esp32 by Espressif Systems. La placa probada se selecciona como ESP32 Dev Module.
3. Instalar PubSubClient, ArduinoJson versión 7 y Adafruit PWM Servo Driver Library con Adafruit BusIO.
4. Abrir el sketch MQTT guardado desde el Arduino IDE de origen, no el sketch de prueba secuencial.
5. Escribir WIFI_SSID, WIFI_PASSWORD y MQTT_HOST de la red nueva.
6. Seleccionar el puerto COM detectado; COM6 era el del equipo original, pero puede cambiar.
7. Usar Upload Speed 115200, Flash Mode DIO y Flash Frequency 40 MHz para las pruebas de traslado.
8. Verificar y subir; esperar escritura o verificación final sin errores. Hard resetting via RTS pin al final es normal si la carga terminó correctamente.
9. Abrir el Monitor serie a 115200 baudios. Comprobar Wi-Fi y MQTT.

Si Arduino IDE no muestra el paquete ESP32, agregar en Preferencias la URL estable https://espressif.github.io/arduino-esp32/package_esp32_index.json y volver al Gestor de placas.

Primero usar SOLO_PRUEBA_MQTT=true con placa solo por USB. Enviar una clasificación y verificar PRUEBA OK con el canal correcto. Este modo no confirma depósitos y no suma puntos. Después de verificar la alimentación, cambiar a false, subir y conectar el PCA9685 y los servos con todo apagado.

11 1 Contrato MQTT del firmware

Orden en smartbin/bin-01/commands/open:

```
{
  "depositId": "UUID_DEL_DEPOSITO",
  "compartment": "plastic",
  "command": "OPEN",
  "timestamp": "FECHA_ISO"
}
```

Confirmación en smartbin/bin-01/events/deposit-confirmed:

```
{
  "depositId": "EL_MISMO_UUID",
  "deviceId": "bin-01",
  "compartment": "plastic",
  "confirmationMode": "TIMED_OPEN_CLOSE"
}
```

El backend actual utiliza depositId para confirmar. El campo confirmationMode describe el ciclo sin sensor. La publicación de confirmación con PubSubClient no constituye un acuse del procesamiento en la base: comprobar también el log del backend y el historial. Los reintentos duraderos y recuperación tras cortes son mejoras posteriores, no garantías del firmware de demostración.

12 Instalación de Flutter y uso en otros dispositivos

12 1 Chrome en la computadora servidor

```
cd C:\SmartBin\practicass\flutter-app
flutter pub get
flutter devices
```

En lib/config/api_config.dart, la rama web debe apuntar a <http://127.0.0.1:3000/api/v1> cuando Chrome y el backend se ejecutan en la misma computadora. Ejecutar flutter run -d chrome y permitir la cámara. Si la cámara falla, utilizar galería y comprobar permisos del navegador. Localhost permite la prueba de cámara; un navegador remoto sobre HTTP puede necesitar HTTPS.

12 2 Android como alternativa

Para el emulador Android estándar se usa <http://10.0.2.2:3000/api/v1>. Para un teléfono físico se usa http://IP_LOCAL_DE_LA_COMPUTADORA:3000/api/v1, con ambos equipos en la misma red. Actualizar la rama Android de api_config.dart: la IP fija de la computadora original no se debe trasladar sin comprobarla.

La variante de teléfono requiere permitir TCP 3000 en la red privada y comprobar permisos de cámara y acceso HTTP de la aplicación Android. Ejecutar flutter devices y elegir el dispositivo. La ruta de Chrome es la opción principal de este manual y debe validarse antes de agregar una segunda variante.

13 Orden de ejecución para cada sesión

La instalación de paquetes y las migraciones no se repiten para cada presentación. PostgreSQL debe estar activo; después ejecutar cuatro terminales independientes en este orden. Los comandos usan la carpeta de ejemplo de destino.

Terminal 1 Broker MQTT

```
cd C:\SmartBin\practicass
& "C:\Program Files\mosquitto\mosquitto.exe" -c ".\mosquitto-local.conf" -v
```

Confirmar que escucha en 1883. Si está funcionando como un servicio con la configuración correcta, no abrir una segunda instancia.

Terminal 2 API de clasificación

```
cd C:\SmartBin\practicas
.\.venv-expo\Scripts\python.exe -m uvicorn app.api.main:app --host 127.0.0.1 --port 8000
```

Esperar carga del modelo y Application startup complete. Para la exposición se omite --reload para evitar reinicios por modificaciones de archivos.

Terminal 3 Backend principal

```
cd C:\SmartBin\practicas\backend-node
npm.cmd start
```

El script start ejecuta node src/server.js. Para desarrollo se puede usar npm.cmd run dev, que observa cambios. Confirmar FastAPI en 8000 y MQTT conectado al broker local.

Terminal 4 Interfaz Flutter

```
cd C:\SmartBin\practicas\flutter-app
flutter run -d chrome
```

Después encender el montaje preparado, comprobar que ESP32 conecte a MQTT y ensayar una clasificación. La alimentación debe permanecer estable antes de enviar órdenes; no encender el interruptor justo después de clasificar. Mantener las terminales abiertas y la computadora sin suspensión.

Comprobaciones de disponibilidad

```
Invoke-RestMethod http://127.0.0.1:8000/api/v1/health
Invoke-RestMethod http://127.0.0.1:3000/api/v1/health
```

El health de Node informa que su proceso HTTP responde; no demuestra por sí solo que PostgreSQL, FastAPI, MQTT y los motores funcionen. La comprobación completa consiste en clasificar, abrir, cerrar, recibir confirmación y consultar los puntos.

14 Manual de operación para la exposición

14 1 Antes de presentar

Preparar una cuenta verificada; comprobar contraseña y sesión. Llevar objetos de plástico, lata, papel y vidrio, y fotografías de respaldo probadas. Confirmar IP del servidor, cámara, etiquetas, recorrido y alimentación. Ejecutar el sistema desde apagado con la red que se usará en la exposición. Conservar una copia privada del sketch, configuración y respaldo de base.

14 2 Demostración

1. Iniciar sesión y mostrar el saldo inicial.

2. Presentar el residuo a la cámara o seleccionar su imagen.
3. Mostrar clase y confianza; explicar que la apertura exige al menos 70 por ciento y material reciclable.
4. Observar que abre únicamente la puerta correspondiente.
5. Esperar cinco segundos y el regreso de la puerta.
6. Comprobar confirmación y abrir nuevamente el historial para ver el depósito y los puntos.
7. Repetir con otro material después de terminar el ciclo.

Explicación sugerida: La aplicación envía una imagen a nuestro backend. Un servicio Python ejecuta el modelo entrenado. El backend aplica reglas y publica una orden MQTT. El ESP32 recibe esa orden, selecciona el canal del PCA9685 y mueve el servo. Al terminar la apertura y cierre, confirma el ciclo y se acreditan puntos.

No afirmar que se verifica físicamente el depósito. El proyecto demuestra clasificación, control conectado y contabilización por ciclo. No presentar como disponibles funciones de sensores, detección de atasco o recuperación automática que todavía no se implementaron.

14 3 Cierre de la sesión

Esperar a que termine cualquier ciclo. Apagar la fuente de los motores. Detener Flutter, Node, FastAPI y Mosquitto manual con Ctrl+C en sus terminales. Desconectar USB antes de desarmar. No desconectar la computadora durante una transacción o restauración de base. PostgreSQL puede mantenerse como servicio si se utilizará nuevamente.

15 Validación de instalación y aceptación

Prueba	Resultado esperado
Importar TensorFlow	Versión impresa sin error de DLL
Health FastAPI	Modelo loaded y respuesta correcta
Health Node	success true y servicio disponible
Broker único	Un proceso escucha en 1883 para la comunicación prevista
MQTT sin motores	PRUEBA recibida por ESP32
Clasificación en modo prueba	Canal correcto sin movimiento ni puntos
Plástico en modo físico	Canal 0 y 10 puntos tras confirmar
Lata en modo físico	Canal 1 y 20 puntos tras confirmar
Papel o cartón	Canal 2 y 5 puntos tras confirmar

Prueba	Resultado esperado
Vidrio	Canal 3 y 15 puntos tras confirmar
Rechazo	Sin apertura con basura o confianza inferior al umbral
Varios ciclos	Sin reinicios y sin abrir puertas equivocadas
Reinicio completo	Reconexión y nueva clasificación aceptada

Registrar fecha, equipo, versiones, red, voltaje medido, parámetros de servos y resultado. El usuario validó el flujo en el equipo original, pero esa validación no reemplaza las pruebas de la computadora nueva y del montaje final.

16 Solución de problemas

Python y aplicación

No module named uvicorn significa que el intérprete elegido no tiene ese paquete. Ejecutar pip y uvicorn con el mismo python.exe del entorno. Para salir de un entorno activado, ejecutar deactivate. La activación no es necesaria cuando se usa la ruta explícita.

Failed to load the native TensorFlow runtime exige comprobar Python de 64 bits, versión compatible, instalación de Visual C++ y la importación aislada de TensorFlow. No atribuir automáticamente el error a falta de AVX por un mensaje genérico. La .venv de Anaconda del origen no funcionó; el Python independiente sí.

Si la app no conecta con la API, revisar baseUrl de Flutter, API Node en 3000 y AI_API_URL en el .env. Si el modelo no está disponible, revisar los dos archivos en modelo y el log de FastAPI. Una ruta copiada de otro equipo no garantiza conectividad.

MQTT y puntos

Puerto 1883 ocupado: identificar procesos y conservar una sola instancia. No lanzar nuevos brokers para intentar resolverlo. Una app que muestra comando publicado no prueba entrega al ESP32; buscar Received PUBLISH y Sending PUBLISH en Mosquitto y Mensaje recibido en Arduino.

PINGREQ y PINGRESP son mensajes normales de mantenimiento. Exceeded timeout significa que el broker dejó de recibir tráfico a tiempo; investigar conectividad, reinicios o bloqueo del programa. Session taken over significa que otra conexión con el mismo identificador sustituyó la anterior, lo que también puede ocurrir al reiniciar.

Si no aumentan los puntos, verificar SOLO_PRUEBA_MQTT=false, mismo depositId en orden y confirmación, backend suscrito al canal correcto y mensaje de confirmación en Node. Volver a consultar historial. Los depósitos creados en modo prueba permanecen pendientes; no se confirman automáticamente al activar los motores.

ESP32 y alimentación

Wrong boot mode: mantener BOOT durante el comienzo de la carga; si hace falta, pulsar y soltar EN sin soltar BOOT, y liberarlo cuando empiece la escritura. Si falla la carga, reducir Upload Speed a 115200 y probar placa sola por USB con otro cable de datos.

No se detecta PCA9685 en 0x40: revisar VCC, GND, SDA, SCL y dirección del módulo. V+ no sustituye VCC. Si el cableado es correcto, utilizar un escáner I2C para identificar la dirección antes de cambiarla en el programa.

Invalid header, tamaños de carga extraños, watchdog o LoadProhibited requieren registrar la salida completa. Si aparecen al encender motores, aislar primero fuente, tierra y cableado; no concluir automáticamente que el firmware o la placa están dañados. Medir la salida del convertidor y probar sin motores, luego con uno. Un LoadProhibited también puede proceder de software y necesita análisis si persiste con alimentación estable.

Si el servo zumba al tope, reducir el recorrido y revisar la carga. Nunca alimentar los cuatro motores desde 3.3 V del ESP32 ni usar el interruptor para desconectar la tierra común. Si la placa falla sola por USB, verificar el programa mínimo y la carga completa antes de reconectar el montaje.

17 Referencias y archivos de consulta

La implementación se consulta en README.md, app/api/main.py, app/api/config.py, app/api/servicios.py, util/modelo.py, backend-node/src/controllers/prediction.controller.js, backend-node/src/services/mqtt.service.js, backend-node/src/services/deposit.service.js, backend-node/src/config/waste.config.js, backend-node/prisma/schema.prisma y flutter-app/lib/config/api_config.dart. El firmware físico debe conservarse desde el sketch MQTT validado en Arduino IDE.

Fuente	Enlace
Instalación de TensorFlow	https://www.tensorflow.org/install/pip
Instalación de Flutter	https://docs.flutter.dev/install/manual
Node.js	https://nodejs.org/en/download
PostgreSQL para Windows	https://www.postgresql.org/download/windows/
Mosquitto	https://mosquitto.org/download/
Soporte Arduino ESP32	https://docs.espressif.com/projects/arduino-esp32/en/latest/installing.html
Errores y carga ESP32	https://docs.espressif.com/projects/esptool/en/latest/esp32/troubleshooting.html
API PubSubClient	https://pubsubclient.knolleary.net/api

Fuente	Enlace
PCA9685 alimentación y uso	https://learn.adafruit.com/16-channel-pwm-servo-driver
Ficha FEETECH FT5325M	https://akizukidenshi.com/goodsaffix/ft5325m_20250613.pdf

Las instrucciones eléctricas y el rango del FT5325M se apoyan en las guías del PCA9685 y en la ficha del fabricante. Consultar siempre la documentación correspondiente a la revisión concreta del hardware antes de modificar alimentación o extremos de recorrido.